

车牌识别

一 实验内容

本实验基于车牌数据集，利用tensorflow框架构建简单的CNN网络，实现车牌识别

二 实验目标

1. 了解车牌数据集
2. 掌握卷积神经网络模型搭建、训练、预测、模型评估的完整流程 **重点**
3. 熟练使用tensorflow.keras构建卷积神经网络 **重点**
4. 熟练利用卷积神经网络解决图像识别任务 **难点**

三 实验环境

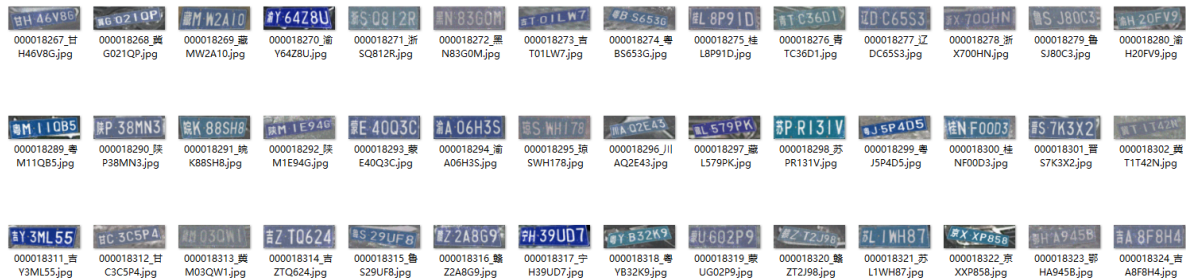
- 操作系统：ubuntu18 及以上
- 语言环境：Python3.6.5
- 编译器：jupyter notebook
- 深度学习环境：TensorFlow2.4.1
- 显卡（GPU）：NVIDIA GeForce RTX 3090

四 实验原理

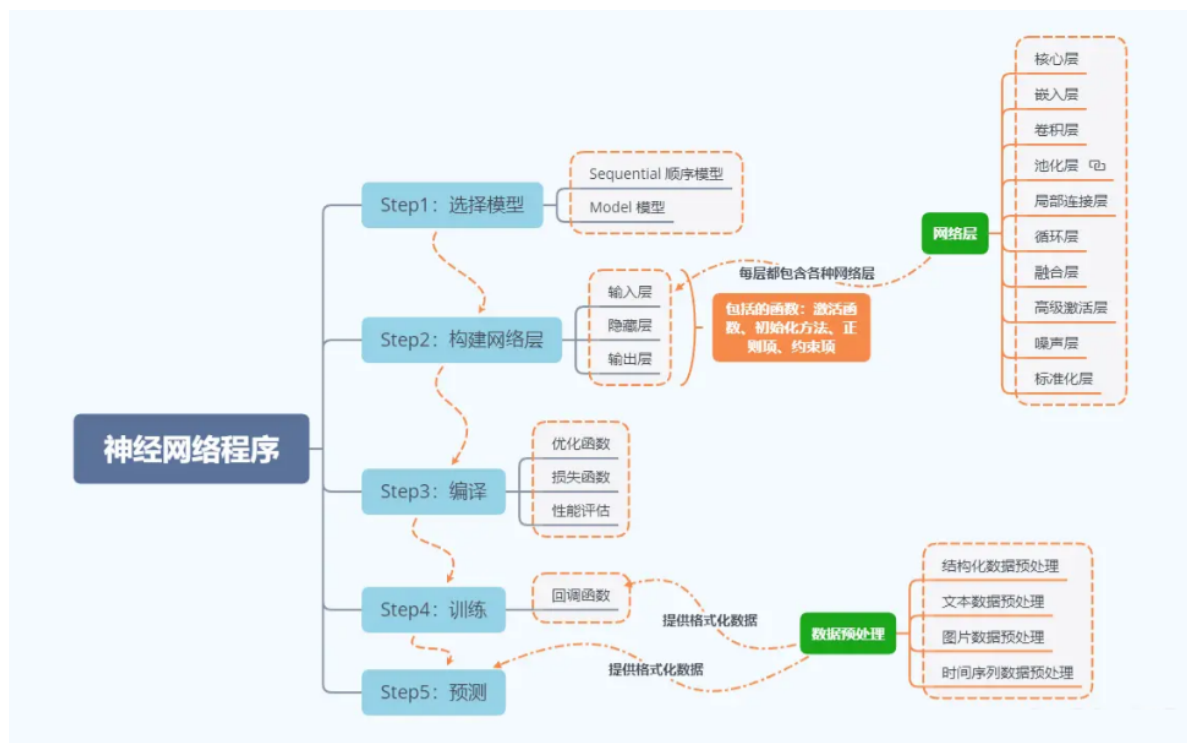
1. 车牌数据集介绍

- licence_plate 车牌数据集图像共包含13675张车牌图像
- 图像大小：226px/72px
- 对应的车牌标签归属地为：

["京", "沪", "津", "渝", "冀", "晋", "蒙", "辽", "吉", "黑", "苏", "浙", "皖", "闽", "赣", "鲁",
"豫", "鄂", "湘", "粤", "桂", "琼", "川", "贵", "云", "藏", "陕", "甘", "青", "宁", "新", "军", "使"]



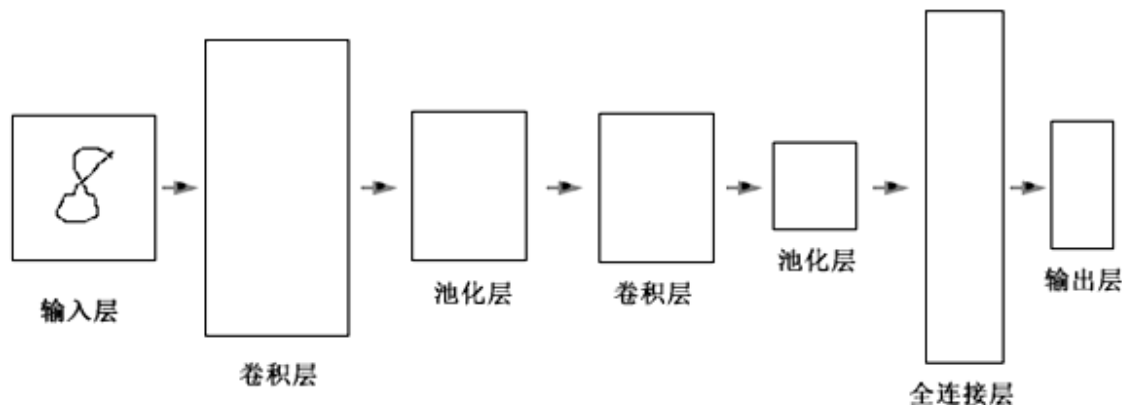
2. 神经网络程序说明



3. 网络结构说明

CNN是用来处理多维数据的多层神经网络。CNN被认为是第一个真正成功的采用多层次结构网络的具有鲁棒性的深度学习方法。CNN通过挖掘数据中的空间上的相关性，来减少网络中的可训练参数的数量，达到改进前向传播网络的反向传播算法效率。CNN中层次之间的紧密联系和空间信息使得其特别适用于图像的处理和理解，并且能够自动的从图像抽取出丰富的相关特性。本实验通过Keras库接口实现CNN网络基础模型的搭建，并且显示网络模型的详细信息。后续我们将用搭建好的模型，借助预处理好的红绿灯图像数据，对其进行训练。最终使用训练好的模型，实现自动驾驶车辆识别红绿灯并且对其做出正确反应的效果。

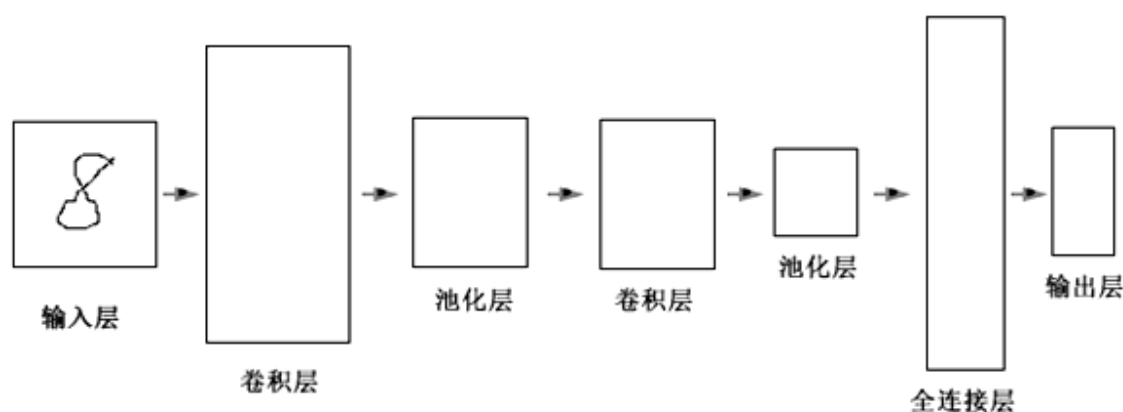
卷积神经网络由输入层、卷积层、池化层、全连接层和输出层组成，其通过逐层处理的方式有效的提取输入数据的特征，尤其针对图像等二维数据输入。通过增加卷积层和池化层，还可以得到更加复杂的网络，其后的全连接层也可以采用多层结构，接下来我们介绍一下卷积神经网络的结构以及每一层的功能。



CNN基本结构

CNN是用来处理多维数据的多层神经网络。CNN被认为是第一个真正成功的采用多层次结构网络的具有鲁棒性的深度学习方法。CNN通过挖掘数据中的空间上的相关性，来减少网络中的可训练参数的数量，达到改进前向传播网络的反向传播算法效率。CNN中层次之间的紧密联系和空间信息使得其特别适用于图像的处理和理解，并且能够自动的从图像中抽取丰富的相关特性。本实验通过Keras库接口实现CNN网络基础模型的搭建，并且显示网络模型的详细信息。后续我们将用搭建好的模型，借助预处理好的红绿灯图像数据，对其进行训练。最终使用训练好的模型，实现自动驾驶车辆识别红绿灯并且对其做出正确反应的效果。

卷积神经网络由输入层、卷积层、池化层、全连接层和输出层组成，其通过逐层处理的方式有效的提取输入数据的特征，尤其针对图像等二维数据输入。通过增加卷积层和池化层，还可以得到更加复杂的网络，其后的全连接层也可以采用多层结构，接下来我们介绍一下卷积神经网络的结构以及每一层的功能。



CNN基本结构

1. 输入层

这一层为数据的输入口，训练数据从这里传到整个卷积神经网络，为下一步卷积操作，提供数据。

2. 卷积层

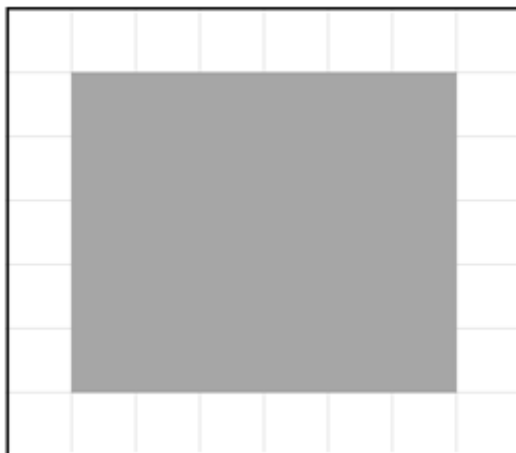
这一层完成的是网络中的卷积操作，可以看做是输入样本和卷积核的内积运算，在进行完卷积操作后，得到的便是特征图。卷积层中是使用同一个卷积核对每一个输入样本进行卷积操作，第二层以及以后的卷积层，是把前一层的特征图作为输入数据，进行同样的卷积操作。如下图：

$$\begin{array}{|c|c|c|} \hline 1 & 6 & 1 \\ \hline 2 & 5 & 8 \\ \hline 4 & 3 & 1 \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline 3 & -2 & -1 \\ \hline -2 & 2 & 1 \\ \hline 1 & 1 & -3 \\ \hline \end{array} = 8$$

该图展示了卷积操作的示意图。上方是一个3x3的输入矩阵（由狗的图片提取的特征图）与一个3x3的卷积核（kernel）进行内积运算，结果为8。下方展示了两张狗的图片，左侧图片尺寸为64*64，右侧图片尺寸为62*62。蓝色线条连接了输入矩阵中的元素与右侧图片中的对应区域，说明了卷积操作是在局部区域上进行的。

卷积操作示意图

对输入样本进行卷积计算后，可以将样本的某些特征更加清晰地表达出来，卷积在信号处理上的意义就是之前的信号衰减后对现在信号的影响，在图像处理的领域中，利用卷积可以将图片中的轮廓特征更明显的表达出来，这是因为图片中物体的轮廓处是色彩变化较大的位置，利用卷积我们可以将色彩变化不那么大的位置（例如背景纹理）模糊化处理，从而凸显出要识别的物体。同一个图片可以使用不同的卷积核处理，就可以得到不同方向上的轮廓特征，如下例中，将输入的图片二值化后得到的像素图用不同的卷积核进行内乘：



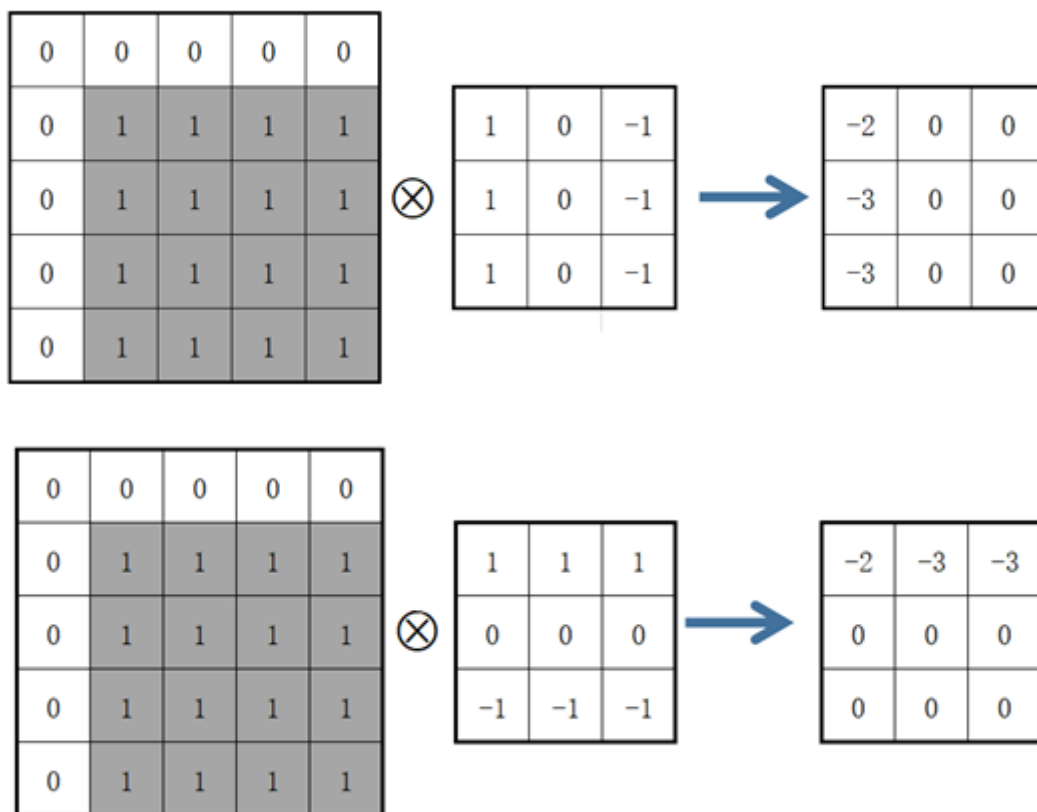
二值化图像

此处为简单示意，将上图左上角起5×5的部分转化为二值化栅格图如下：

0	0	0	0	0
0	1	1	1	1
0	1	1	1	1
0	1	1	1	1
0	1	1	1	1

5*5的二值化图像

分别与两个卷积核内乘结果如下：



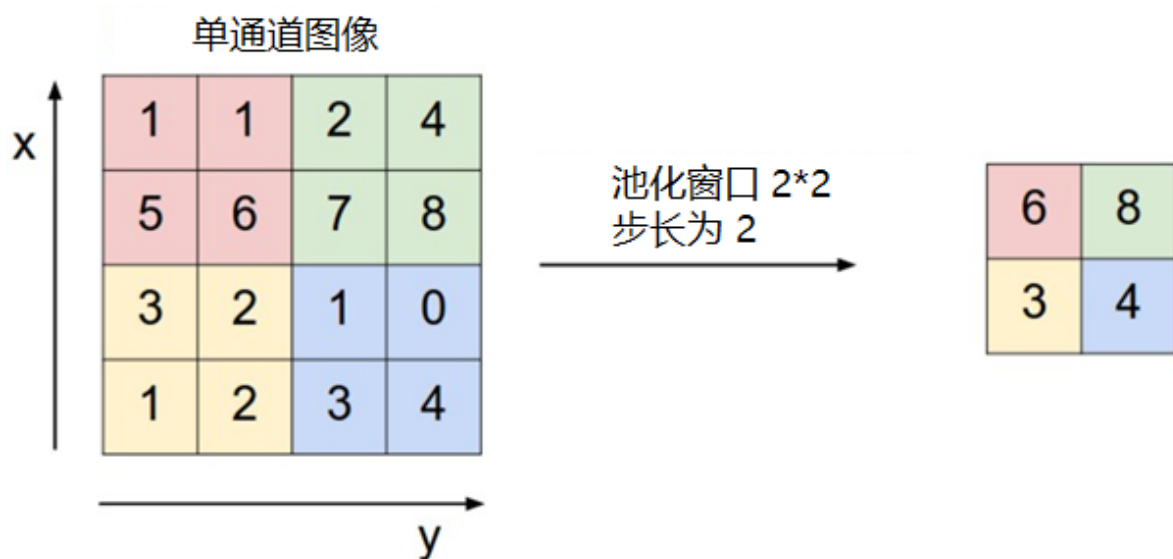
卷积操作

上面的两个卷积核的功能就是得到纵向轮廓和横向轮廓，可以从结果看出，水平方向的边界和垂直方向的边界都在边沿的3×3位置以内，这就是卷积对轮廓识别的功能，通过卷积运算，我们可以得到更清晰的边界数据来识别物体，特别是对三通道的彩色图片，分别对R (red) G (green) B (blue) 三色进行卷积运算，能得到对计算机而言辨识度更高的处理后图像。

我们可以发现，卷积内乘后图像尺寸会发生变化，对64×64的输入样本使用3×3的卷积核进行卷积运算后，得到62×62的特征图。特征图的尺寸会小于输入样本，为了得到与输入样本尺寸相同的特征图，可以使用0补充样本边界。上面的示意图卷积核的滑动步长为1，步长越大，卷积后的特征图越小。一个卷积层也可以有多个不同的卷积核，每个卷积核都对应一个特征图。此外，卷积结果不能直接作为特征图，需要通过激活函数运算后，把函数输出结果作为特征图。常见的激活函数包括sigmoid、tanh、ReLU等函数，这些函数都是非线性的函数，这里使用非线性函数是为了能表达能复杂的分类边界，如果不使用非线性函数，图像的卷积运算无非是矩阵的相乘变换，不具备分别复杂物体的能力，也就是说，线性函数的组合在高维空间并不能很好的对样本特征进行分割。

3. 池化层

池化层的作用是减小卷积层产生的特征图的尺寸。选择一个区域，根据该区域的特征图得到新特征图的这个过程称为池化。主要的池化方法有最大池化、平均池化、求和池化等，其中最常用的是最大池化，最大池化是选取图像区域内的最大值作为新的特征图，如下图所示：



池化层示意图

4. 全连接层

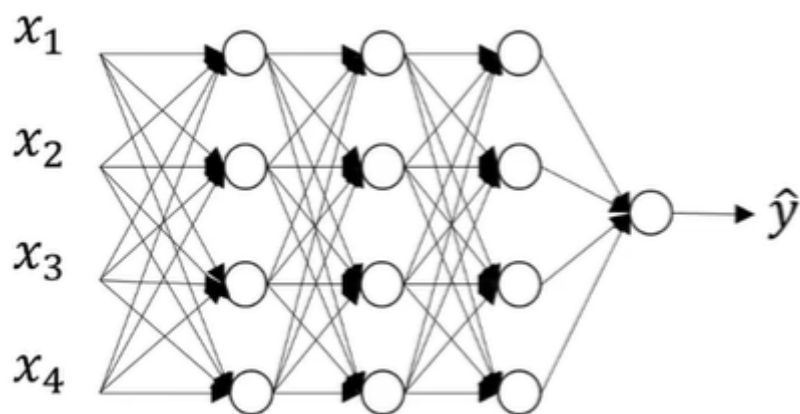
全连接层的功能是连接所有的特征，将输出值送给分类器（如softmax分类器），图像经过卷积层和池化层后得到的轮廓数据会在这一层得到判断，根据轮廓特征可以识别出各种结果的可能概率。

5. 输出层

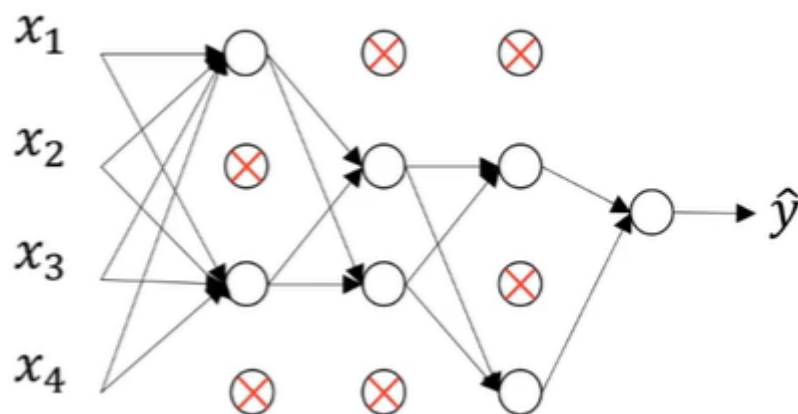
卷积神经网络的输出层是使用似然函数计算各类别的似然概率。使用softmax函数可以计算输出单元的似然概率，然后把概率最大的数字作为分类结果输出。

6. Dropout层

Dropout做的就是以一定的概率将每个隐层神经元的输出设置为零。以这种方式“dropped out”的神经元既不参与前向传播，也不参与反向传播。所以每次提出一个输入，该神经网络就尝试一个不同的结构，但是所有这些结构之间共享权重。因为神经元不能依赖于其他特定神经元而存在，所以这种技术降低了神经元复杂的互适应关系。正因如此，要被迫学习更为鲁棒的特征，这些特征在结合其他神经元的一些不同随机子集时有用。



原始神经网络

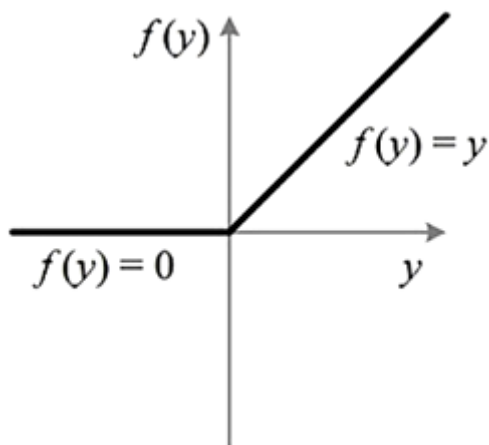


添加了Dropout层的神经网络

在训练时，随机选择部分神经元，使其置零，不参与本次优化迭代。每次减少参与优化迭代的神经元数目，使有效网络变小，防止过拟合，增强模型的泛化能力。

7. 非线性激活函数ReLU

在本实验中，我们使用非线性激活函数ReLU，ReLU函数图像如下：



ReLU图像

ReLU函数形式如下：

$$\text{ReLU} = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

ReLU函数

从ReLU的图像可以看出，ReLU是分段线性函数，所有负值的函数值均为0，而正值得函数值不变，这种操作叫做单侧抑制，单侧抑制可以让网络中的神经元具有稀疏激活性，这样有利于模型更好的挖掘相关特征，拟合数据。使用ReLU激活函数，使得网络训练速度提高，同时解决了训练较深的网络时出现的一些问题，如梯度弥散问题等。

五 实验步骤

一、前期准备

设置GPU（如果使用的是CPU可以忽略这步）

```
import tensorflow as tf

gpus = tf.config.list_physical_devices("GPU")

if gpus:
    tf.config.experimental.set_memory_growth(gpus[0], True) #设置GPU显存用量按需使用
    tf.config.set_visible_devices([gpus[0]], "GPU")
```

导入数据

```
import matplotlib.pyplot as plt
# 支持中文
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

import os,PIL,random,pathlib

# 设置随机种子尽可能使结果可以重现
import numpy as np
np.random.seed(1)

# 设置随机种子尽可能使结果可以重现
import tensorflow as tf
tf.random.set_seed(1)
```

```
import matplotlib.pyplot as plt
# 支持中文
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

import os,PIL,random,pathlib

# 设置随机种子尽可能使结果可以重现
import numpy as np
np.random.seed(1)

# 设置随机种子尽可能使结果可以重现
import tensorflow as tf
tf.random.set_seed(1)
```

输出

```
['../dataset/licence_plate/000000000_川W9BR26.jpg',
 '../dataset/licence_plate/000000000_藏WP66B0.jpg',
 '../dataset/licence_plate/000000001_沪E264UD.jpg']
```

```
image_count = len(list(pictures_paths))

print("图片总数为: ",image_count)
```


输出结果:

图片总数为: 13056

```
# 获取数据标签
all_label_names = [path.split("_")[-1].split(".")[0] for path in pictures_paths]
all_label_names[:3]
```

数据可视化图片

```
plt.figure(figsize=(10,5))
plt.suptitle("数据示例",fontsize=15)

for i in range(20):
    plt.subplot(5,4,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)

    # 显示图片
    images = plt.imread(pictures_paths[i])
    plt.imshow(images)
    # 显示标签
    plt.xlabel(all_label_names[i],fontsize=13)

plt.show()
```

数据示例



标签数字化

```
char_enum =
["京", "沪", "津", "渝", "冀", "晋", "蒙", "辽", "吉", "黑", "苏", "浙", "皖", "闽", "赣", "鲁", \
"豫", "鄂", "湘", "粤", "桂", "琼", "川", "贵", "云", "藏", "陕", "甘", "青", "宁", "新", "军", "使"]

number = [str(i) for i in range(0, 10)] # 0 到 9 的数字
alphabet = [chr(i) for i in range(65, 91)] # A 到 Z 的字母
```

```

char_set      = char_enum + number + alphabet
char_set_len  = len(char_set)
label_name_len = len(all_label_names[0])

# 将字符串数字化
def text2vec(text):
    vector = np.zeros([label_name_len, char_set_len])
    for i, c in enumerate(text):
        idx = char_set.index(c)
        vector[i][idx] = 1.0
    return vector

all_labels = [text2vec(i) for i in all_label_names]

```

二、构建一个tf.data.Dataset

1. 预处理函数

```

def preprocess_image(image):
    image = tf.image.decode_jpeg(image, channels=1)
    image = tf.image.resize(image, [50, 200])
    return image/255.0

def load_and_preprocess_image(path):
    image = tf.io.read_file(path)
    return preprocess_image(image)

```

2. 加载数据

构建 `tf.data.Dataset` 最简单的方法就是使用 `from_tensor_slices` 方法。

```

AUTOTUNE = tf.data.experimental.AUTOTUNE

path_ds = tf.data.Dataset.from_tensor_slices(pictures_paths)
image_ds = path_ds.map(load_and_preprocess_image, num_parallel_calls=AUTOTUNE)
label_ds = tf.data.Dataset.from_tensor_slices(all_labels)

image_label_ds = tf.data.Dataset.zip((image_ds, label_ds))
image_label_ds

```

输出结果:

```
<ZipDataset shapes: ((50, 200, 1), (7, 69)), types: (tf.float32, tf.float64)>
```

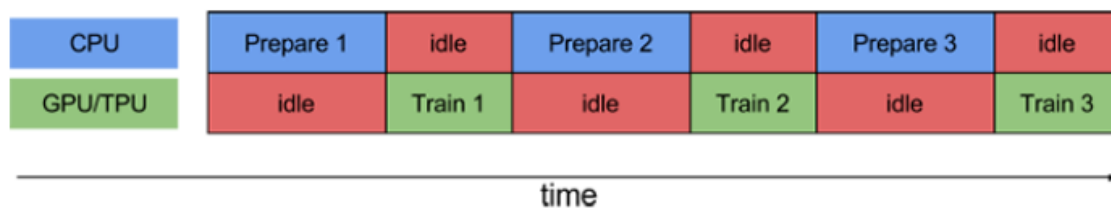
```

train_ds = image_label_ds.take(5000).shuffle(5000) # 前1000个batch
val_ds   = image_label_ds.skip(5000).shuffle(1000) # 跳过前1000，选取后面的

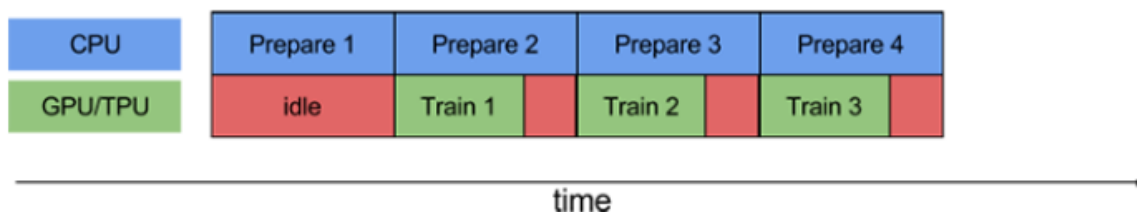
```

3. 配置数据

先复习一下 `prefetch()` 函数。`prefetch()` 功能详细介绍：CPU 正在准备数据时，加速器处于空闲状态。相反，当加速器正在训练模型时，CPU 处于空闲状态。因此，训练所用的时间是 CPU 预处理时间和加速器训练时间的总和。`prefetch()` 将训练步骤的预处理和模型执行过程重叠到一起。当加速器正在执行第 N 个训练步时，CPU 正在准备第 N+1 步的数据。这样做不仅可以最大限度地缩短训练的单步用时（而不是总用时），而且可以缩短提取和转换数据所需的时间。如果不使用 `prefetch()`，CPU 和 GPU/TPU 在大部分时间都处于空闲状态：



使用 `prefetch()` 可显著减少空闲时间：



```
BATCH_SIZE = 16

train_ds = train_ds.batch(BATCH_SIZE)
train_ds = train_ds.prefetch(buffer_size=AUTOTUNE)

val_ds = val_ds.batch(BATCH_SIZE)
val_ds = val_ds.prefetch(buffer_size=AUTOTUNE)
val_ds
```

输出：

```
<PrefetchDataset shapes: ((None, 50, 200, 1), (None, 7, 69)), types:
(tf.float32, tf.float64)>
```

三、搭建网络模型

目前这里主要是带大家跑通代码、整理一下思路，大家可以自行优化网络结构、调整模型参数。

```
from tensorflow.keras import datasets, layers, models

model = models.Sequential([

    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(50, 200, 1)), #卷积
    layers.MaxPooling2D((2, 2)), #池化层1, 2*2采样
    layers.Conv2D(64, (3, 3), activation='relu'), #卷积层2, 卷积核3*3
    layers.MaxPooling2D((2, 2)), #池化层2, 2*2采样
```

```

layers.Flatten(),
# layers.Dense(1000, activation='relu'),
layers.Dense(1000, activation='relu'),
layers.Dropout(0.3),
layers.Dense(label_name_len * char_set_len),
layers.Reshape([label_name_len, char_set_len]),
layers.Softmax()
])
# 打印网络结构
model.summary()

```

#Flatten层，连接卷积层与全连接层
#全连接层，特征进一步提取
#全连接层，特征进一步提取
#输出层，输出预期结果

输出结果:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 48, 198, 32)	320
max_pooling2d (MaxPooling2D)	(None, 24, 99, 32)	0
conv2d_1 (Conv2D)	(None, 22, 97, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 11, 48, 64)	0
flatten (Flatten)	(None, 33792)	0
dense (Dense)	(None, 1000)	33793000
dropout (Dropout)	(None, 1000)	0
dense_1 (Dense)	(None, 483)	483483
reshape (Reshape)	(None, 7, 69)	0
softmax (Softmax)	(None, 7, 69)	0
Total params: 34,295,299		
Trainable params: 34,295,299		
Non-trainable params: 0		

四、设置动态学习率

这里先罗列一下学习率大与学习率小的优缺点。

- 学习率大
 - 优点：1、加快学习速率。2、有助于跳出局部最优值。
 - 缺点：1、导致模型训练不收敛。2、单单使用大学习率容易导致模型不精确。
- 学习率小
 - 优点：1、有助于模型收敛、模型细化。2、提高模型精度。
 - 缺点：1、很难跳出局部最优值。2、收敛缓慢。

注意：这里设置的动态学习率为：指数衰减型（ExponentialDecay）。在每一个epoch开始前，学习率（learning_rate）都将会重置为初始学习率（initial_learning_rate），然后再重新开始衰减。计算公式如下：

```
learning_rate = initial_learning_rate * decay_rate ^ (step / decay_steps)
```

```
# 设置初始学习率
initial_learning_rate = 1e-3

lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
    initial_learning_rate,
    decay_steps=50,          # 敲黑板!!! 这里是指 steps, 不是指epochs
    decay_rate=0.96,        # lr经过一次衰减就会变成 decay_rate*lr
    staircase=True)

# 将指数衰减学习率送入优化器
optimizer = tf.keras.optimizers.Adam(learning_rate=lr_schedule)
```

五、编译

```
model.compile(optimizer=optimizer,
               loss='categorical_crossentropy',
               metrics=['accuracy'])
```

六、训练

```
epochs = 50

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=epochs
)
```

输出结果

```
Epoch 1/50
313/313 [=====] - 9s 17ms/step - loss: 3.5249 -
accuracy: 0.0537 - val_loss: 3.3293 - val_accuracy: 0.0610
Epoch 2/50
313/313 [=====] - 5s 14ms/step - loss: 3.3078 -
accuracy: 0.0769 - val_loss: 3.1374 - val_accuracy: 0.2068
Epoch 3/50
313/313 [=====] - 5s 14ms/step - loss: 2.8676 -
accuracy: 0.2455 - val_loss: 2.4188 - val_accuracy: 0.3579
Epoch 4/50
313/313 [=====] - 5s 14ms/step - loss: 2.1170 -
accuracy: 0.4031 - val_loss: 2.0383 - val_accuracy: 0.4391
Epoch 5/50
```

```
313/313 [=====] - 5s 14ms/step - loss: 1.6881 -  
accuracy: 0.5104 - val_loss: 1.8287 - val_accuracy: 0.4879  
Epoch 6/50  
313/313 [=====] - 5s 14ms/step - loss: 1.3931 -  
accuracy: 0.5900 - val_loss: 1.7230 - val_accuracy: 0.5100  
Epoch 7/50  
313/313 [=====] - 5s 14ms/step - loss: 1.2091 -  
accuracy: 0.6429 - val_loss: 1.6496 - val_accuracy: 0.5278  
Epoch 8/50  
313/313 [=====] - 5s 14ms/step - loss: 1.0908 -  
accuracy: 0.6726 - val_loss: 1.6069 - val_accuracy: 0.5420  
Epoch 9/50  
313/313 [=====] - 5s 14ms/step - loss: 0.9867 -  
accuracy: 0.7034 - val_loss: 1.5799 - val_accuracy: 0.5508  
Epoch 10/50  
313/313 [=====] - 5s 14ms/step - loss: 0.9207 -  
accuracy: 0.7240 - val_loss: 1.5607 - val_accuracy: 0.5562  
Epoch 11/50  
313/313 [=====] - 5s 14ms/step - loss: 0.8628 -  
accuracy: 0.7378 - val_loss: 1.5463 - val_accuracy: 0.5590  
Epoch 12/50  
313/313 [=====] - 5s 14ms/step - loss: 0.8290 -  
accuracy: 0.7463 - val_loss: 1.5352 - val_accuracy: 0.5638  
Epoch 13/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7932 -  
accuracy: 0.7594 - val_loss: 1.5304 - val_accuracy: 0.5653  
Epoch 14/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7778 -  
accuracy: 0.7665 - val_loss: 1.5204 - val_accuracy: 0.5675  
Epoch 15/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7691 -  
accuracy: 0.7686 - val_loss: 1.5163 - val_accuracy: 0.5691  
Epoch 16/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7548 -  
accuracy: 0.7743 - val_loss: 1.5180 - val_accuracy: 0.5691  
Epoch 17/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7325 -  
accuracy: 0.7772 - val_loss: 1.5137 - val_accuracy: 0.5703  
Epoch 18/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7458 -  
accuracy: 0.7791 - val_loss: 1.5128 - val_accuracy: 0.5706  
Epoch 19/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7280 -  
accuracy: 0.7826 - val_loss: 1.5125 - val_accuracy: 0.5716  
Epoch 20/50  
313/313 [=====] - 5s 15ms/step - loss: 0.7317 -  
accuracy: 0.7800 - val_loss: 1.5118 - val_accuracy: 0.5709  
Epoch 21/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7373 -  
accuracy: 0.7743 - val_loss: 1.5100 - val_accuracy: 0.5714  
Epoch 22/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7321 -  
accuracy: 0.7794 - val_loss: 1.5100 - val_accuracy: 0.5718  
Epoch 23/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7114 -  
accuracy: 0.7857 - val_loss: 1.5097 - val_accuracy: 0.5722  
Epoch 24/50
```

```
313/313 [=====] - 5s 14ms/step - loss: 0.7157 -  
accuracy: 0.7852 - val_loss: 1.5090 - val_accuracy: 0.5725  
Epoch 25/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7084 -  
accuracy: 0.7870 - val_loss: 1.5094 - val_accuracy: 0.5725  
Epoch 26/50  
313/313 [=====] - 5s 15ms/step - loss: 0.6839 -  
accuracy: 0.7925 - val_loss: 1.5090 - val_accuracy: 0.5725  
Epoch 27/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7237 -  
accuracy: 0.7813 - val_loss: 1.5088 - val_accuracy: 0.5723  
Epoch 28/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7144 -  
accuracy: 0.7836 - val_loss: 1.5088 - val_accuracy: 0.5723  
Epoch 29/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7191 -  
accuracy: 0.7859 - val_loss: 1.5084 - val_accuracy: 0.5725  
Epoch 30/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7230 -  
accuracy: 0.7783 - val_loss: 1.5085 - val_accuracy: 0.5724  
Epoch 31/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7224 -  
accuracy: 0.7820 - val_loss: 1.5086 - val_accuracy: 0.5725  
Epoch 32/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7028 -  
accuracy: 0.7880 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 33/50  
313/313 [=====] - 5s 15ms/step - loss: 0.7148 -  
accuracy: 0.7831 - val_loss: 1.5085 - val_accuracy: 0.5725  
Epoch 34/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7091 -  
accuracy: 0.7861 - val_loss: 1.5084 - val_accuracy: 0.5724  
Epoch 35/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7185 -  
accuracy: 0.7867 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 36/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7117 -  
accuracy: 0.7835 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 37/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7224 -  
accuracy: 0.7842 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 38/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7155 -  
accuracy: 0.7847 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 39/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7121 -  
accuracy: 0.7826 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 40/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7237 -  
accuracy: 0.7809 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 41/50  
313/313 [=====] - 5s 14ms/step - loss: 0.7206 -  
accuracy: 0.7810 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 42/50  
313/313 [=====] - 5s 14ms/step - loss: 0.6994 -  
accuracy: 0.7927 - val_loss: 1.5084 - val_accuracy: 0.5727  
Epoch 43/50
```

```

313/313 [=====] - 5s 14ms/step - loss: 0.7075 -
accuracy: 0.7840 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 44/50
313/313 [=====] - 5s 14ms/step - loss: 0.7179 -
accuracy: 0.7842 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 45/50
313/313 [=====] - 5s 14ms/step - loss: 0.7017 -
accuracy: 0.7876 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 46/50
313/313 [=====] - 5s 14ms/step - loss: 0.6971 -
accuracy: 0.7900 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 47/50
313/313 [=====] - 5s 14ms/step - loss: 0.7054 -
accuracy: 0.7884 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 48/50
313/313 [=====] - 5s 14ms/step - loss: 0.7107 -
accuracy: 0.7880 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 49/50
313/313 [=====] - 5s 14ms/step - loss: 0.7169 -
accuracy: 0.7818 - val_loss: 1.5084 - val_accuracy: 0.5727
Epoch 50/50
313/313 [=====] - 5s 14ms/step - loss: 0.7033 -
accuracy: 0.7918 - val_loss: 1.5084 - val_accuracy: 0.5727

```

七、模型评估

```

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

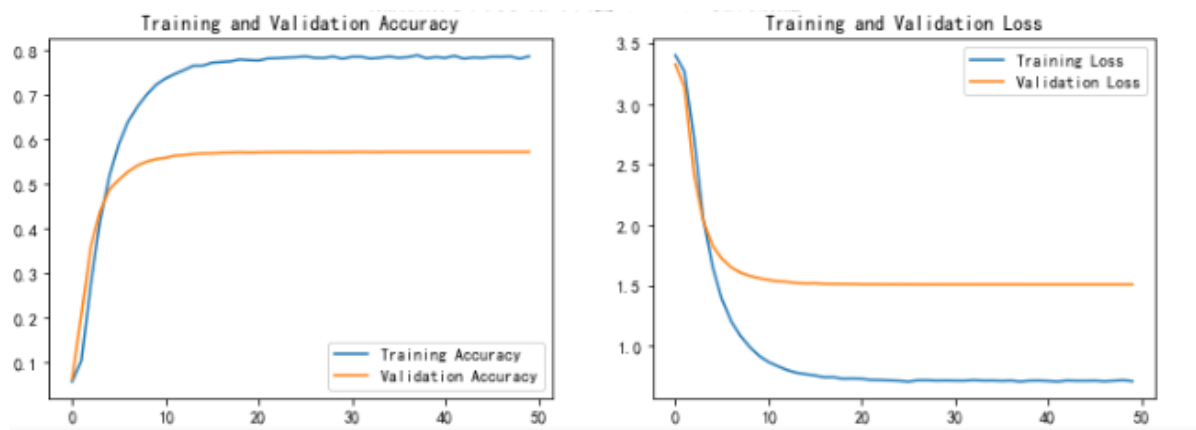
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.suptitle("License Plate Recognition", fontsize=12)

plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```

输出结果:



八、保存和加载模型

```
# 保存模型
model.save('model/15_model.h5')
# 加载模型
new_model = tf.keras.models.load_model('model/15_model.h5')
```

九、预测

```
def vec2text(vec):
    """
    还原标签（向量->字符串）
    """
    text = []
    for i, c in enumerate(vec):
        text.append(char_set[c])
    return "".join(text)

plt.figure(figsize=(10, 8)) # 图形的宽为10高为8
plt.suptitle("微信公众号(K同学啊)中回复(DL+15)可获取数据", fontsize=15)

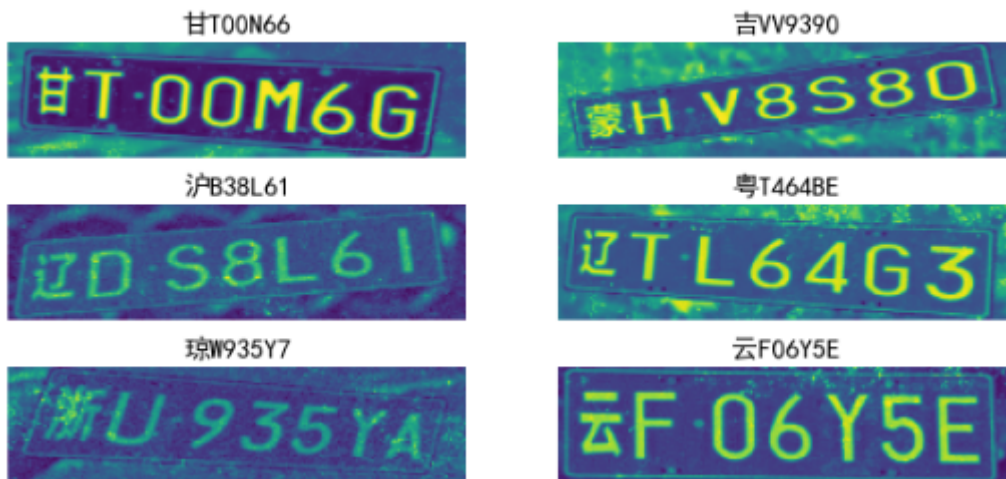
for images, labels in val_ds.take(1):
    for i in range(6):
        ax = plt.subplot(5, 2, i + 1)
        # 显示图片
        plt.imshow(images[i])

        # 需要给图片增加一个维度
        img_array = tf.expand_dims(images[i], 0)

        # 使用模型预测验证码
        predictions = model.predict(img_array)
        plt.title(vec2text(np.argmax(predictions, axis=2)[0]), fontsize=15)

    plt.axis("off")
```

输出结果:



六 实验总结

CNN灵活应用与解决图像识别问题，完整使用流程如下：

